

Architecture et organisation du code

Dans une app Angular moderne, on place les services à portée globale dans un **CoreModule** importé une seule fois dans `AppModule`, et on sépare les fonctionnalités en modules (par ex. un module `AuthModule` pour l'UI de login). Par exemple :

- `src/app/core/auth/auth.service.ts` - `AuthService` (singleton) avec `providedIn: 'root'`.
- `src/app/core/interceptors/auth.interceptor.ts` - `AuthInterceptor` enregistré dans les providers (`HTTP_INTERCEPTORS`).
- `src/app/core/guards/auth.guard.ts` - **AuthGuard** (optionnel) pour protéger les routes (`CanActivate` renvoyant `authService.isLoggedIn()`).
- `src/app/shared/` - composants partagés (ex. formulaires Angular Material).

Cette séparation suit les recommandations Angular : un **AuthService** centralise la logique d'authentification, ce qui rend le code modulaire et facile à maintenir ¹. Par exemple, tous les appels HTTP d'authentification passent par `AuthService`, ce qui évite la duplication.

AuthService : login et stockage des tokens

Le `AuthService` contient une méthode `login(email, password)` qui utilise `HttpClient` pour appeler `/api/users/login`. On passe `{ withCredentials: true }` si le backend utilise des cookies `HttpOnly` pour le refresh-token. La réponse (JSON) contient typiquement l'**access token** (JWT court-vie) et éventuellement un **refresh token**. Dans l'extrait ci-dessous, on suppose que le backend renvoie `{ accessToken, refreshToken }`. On utilise RxJS `pipe`, `tap`, `catchError` pour gérer la réponse et les erreurs ; on peut également utiliser `shareReplay(1)` pour éviter d'émettre plusieurs requêtes en cas de multiples souscriptions ².

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private accessToken: string | null = null;

  constructor(private http: HttpClient, private router: Router) {}

  /** Connexion : envoie email/mot de passe, récupère jetons */
  login(email: string, password: string): Observable<any> {
    return this.http.post<{ accessToken: string, refreshToken: string }>('/api/users/login',
      { email, password }, { withCredentials: true })
      .pipe(
        // partager l'Observable pour éviter plusieurs appels au même login
        shareReplay(1),
        tap(res => {
          // stocker en mémoire l'access token pour le header HTTP
          this.accessToken = res.accessToken;
          // (le refresh token est stocké en cookie HttpOnly par le serveur)
        })
      );
  }
}
```

```

    }},
    catchError(err => {
      console.error('Erreur de login', err);
      // Propager l'erreur pour affichage éventuel
      throw err;
    })
  );
}

/** Getter du token pour l'intercepteur */
getAccessToken(): string | null {
  return this.accessToken;
}

/** Logout (effacer état et rediriger) */
logout(): void {
  this.accessToken = null;
  // (éventuellement appeler /api/auth/logout pour effacer cookie refresh)
  this.router.navigate(['/login']);
}
}

```

Explications : `AuthService` est un singleton (`providedIn: 'root'`). La méthode `login()` renvoie un Observable issu de `HttpClient.post`. On utilise `shareReplay(1)` pour partager la même requête si plusieurs composants s'abonnent en même temps ². En cas de succès, on stocke l'`accessToken` (par exemple en mémoire) et on redirige l'utilisateur. Le **refresh token** n'est *pas* stocké en clair : on laisse le backend le mettre dans un cookie `HttpOnly` (voir ci-dessous). En cas d'erreur (401, 400...), on la remonte via `catchError`.

AuthInterceptor : ajouter l'en-tête et gérer le rafraîchissement

Un `HttpInterceptor` global intercepte toutes les requêtes sortantes pour y ajouter le header **Authorization: Bearer** avec l'access token (si présent). Par exemple, la doc Angular montre un interceptor type qui clone la requête et fixe le header `Authorization` via `authService.getAccessToken()` ³. Dans la pratique, l'intercepteur inclut aussi une gestion des erreurs 401 pour rafraîchir le token :

```

@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  private isRefreshing = false;
  private refreshTokenSubject = new BehaviorSubject<string | null>(null);

  constructor(private auth: AuthService, private http: HttpClient, private router: Router) {}

  intercept(req: HttpRequest<any>, next: HttpHandler):
  Observable<HttpEvent<any>> {
    // Ajouter le header Authorization si on a un access token
    let authReq = req;

```

```

    const token = this.auth.getAccessToken();
    if (token) {
      authReq = req.clone({ setHeaders: { Authorization: `Bearer ${token}`
    } }));
    }
    return next.handle(authReq).pipe(
      catchError(err => {
        if (err.status === 401) {
          // Si 401, tenter de rafraîchir le token
          return this.handle401Error(authReq, next);
        }
        return throwError(err);
      })
    );
  }

  private handle401Error(request: HttpRequest<any>, next: HttpHandler) {
    if (!this.isRefreshing) {
      this.isRefreshing = true;
      this.refreshTokenSubject.next(null);
      // Appel au backend /api/auth/refresh (avec le cookie HttpOnly
      automatique)
      return this.http.post<{ accessToken: string }>('/api/auth/refresh', {},
        { withCredentials: true }).pipe(
        switchMap((res: any) => {
          this.isRefreshing = false;
          // Mettre à jour l'access token en mémoire
          this.auth.accessToken = res.accessToken;
          this.refreshTokenSubject.next(res.accessToken);
          // Rejouer la requête échouée avec le nouveau token
          return next.handle(request.clone({ setHeaders: { Authorization:
`Bearer ${res.accessToken}` } }));
        })),
        catchError((refreshErr) => {
          this.isRefreshing = false;
          // Si échec (ex : refresh token invalide), logout et rediriger
          this.auth.logout();
          return throwError(refreshErr);
        })
      );
    } else {
      // Si un rafraîchissement est déjà en cours, attendre son résultat
      return this.refreshTokenSubject.pipe(
        filter(t => t !== null),
        take(1),
        switchMap((t) => next.handle(request.clone({ setHeaders: {
Authorization: `Bearer ${t}` } })))
      );
    }
  }
}

```

Dans ce code, on injecte `AuthService` pour lire l'access token en mémoire. Sur une erreur 401, on appelle `/api/auth/refresh` (qui prend le *refresh token* depuis le cookie `HttpOnly`)⁴. Ce mécanisme suit les exemples courants : on n'envoie le refresh qu'une fois (grâce à `isRefreshing/BehaviorSubject`), et on rejoue la requête initiale avec le nouveau token. L'intercepteur gère aussi la redirection vers la page de login en cas d'échec de rafraîchissement. Ce schéma (cloner la requête et ajouter les headers) est présenté dans la doc Angular³ et dans de nombreux tutoriels modernes d'Angular.

Cookies `HttpOnly` et stockage des jetons

Pour améliorer la sécurité, on conseille de **ne pas exposer les jetons** (tokens) au JavaScript côté client (éviter `localStorage/sessionStorage` pour les tokens sensibles). À la place, on peut laisser le backend exprès renvoyer le **refresh token** dans un cookie `HttpOnly` (non lisible par JS) avec `Secure: true` et `SameSite` approprié⁵⁶. Par exemple, un login réussi peut faire (côté Express) :

```
// Exemple .NET/C# : ajouter cookie HttpOnly depuis le backend
Response.Cookies.Append("refreshToken", token, new CookieOptions {
    HttpOnly = true,
    Secure = true,
    SameSite = SameSiteMode.None, // ou Strict/Lax selon le contexte
    Expires = DateTime.UtcNow.AddDays(7)
});
```

Avantages : un cookie `HttpOnly` n'est pas accessible par les scripts (protection XSS)⁶. Le navigateur l'envoie automatiquement sur chaque requête vers la même origine, ce qui évite d'avoir à inclure explicitement le token dans le corps ou les headers. La configuration `Secure` (HTTPS uniquement) et `SameSite=None` (pour cross-site ou sous-domaines) est recommandée⁵⁷.

En pratique, on stocke généralement *seulement* le **refresh token** en cookie `HttpOnly`, et on conserve l'**access token** court-terme en mémoire (par ex. dans `AuthService.accessToken`). Comme le note une réponse StackOverflow, « Your access token doesn't need to be stored anywhere like local/session storage or cookie. You can simply keep it in some SPA service... If the page is reloaded, you just rotate tokens during initial load (remember `HttpOnly` cookie is stuck to your domain)⁸. » Autrement dit, en cas de reload on appelle le endpoint `/api/auth/refresh` pour obtenir un nouvel `accessToken` (grâce au cookie), sans avoir stocké quoi que ce soit de sensible en `localStorage`⁸. Cette approche est maintenant recommandée par la communauté pour éviter les failles XSS.

Sécurité des jetons (XSS, CSRF, etc.)

- **XSS (Cross-Site Scripting)** : Angular désinfecte (sanitise) automatiquement les valeurs liées dans les templates (interpolation HTML, liens, styles, etc.) pour réduire les risques XSS⁹. En complément, **éviter de stocker les tokens dans le DOM**. L'usage de cookies `HttpOnly` empêche tout vol de token via du script malveillant⁶.
- **CSRF (Cross-Site Request Forgery)** : stocker les tokens dans un cookie (même `HttpOnly`) expose théoriquement à des attaques CSRF, car le navigateur envoie le cookie automatiquement. Angular fournit toutefois un mécanisme intégré de protection CSRF. En important `HttpClientXsrfModule` (inclus par défaut dans `HttpClientModule`), Angular s'attend à ce

que le serveur envoie un cookie (nom par défaut `XSRF-TOKEN`) contenant un jeton CSRF. Angular lira alors ce cookie et ajoutera automatiquement un header `X-XSRF-TOKEN` dans chaque requête mutante (POST/PUT/DELETE) ¹⁰ ¹¹. Le serveur doit alors valider ce jeton. Ce **double-submit cookie** (cookie + header) assure qu'un site tiers ne peut pas induire involontairement des requêtes valides. En pratique, on configure souvent le serveur pour qu'il émette un cookie CSRF (`XSRF-TOKEN`) sur le login, puis Angular gère le header. Comme le résume le guide de sécurité Angular : « The server sends a CSRF token in a cookie... Angular reads this cookie and automatically adds it to a custom header (X-XSRF-TOKEN) » ¹¹. En bonus, le cookie de session `refreshToken` doit impérativement être `SameSite` strict ou `Lax` si possible, ou on utilise explicitement ce mécanisme XSRF pour se prémunir.

- **Autres (CSP, HTTPS, etc.)** : Il est fortement recommandé de servir l'app en **HTTPS** et de fixer un **Content Security Policy** strict pour bloquer l'exécution de scripts non autorisés. Dans le cadre d'Angular/Material, veillez à suivre les bonnes pratiques de développement (ne pas manipuler le DOM manuellement via `ElementRef` ou `innerHTML` sans sanitization, utiliser `Renderer2` si nécessaire ¹², utiliser les bindings Angular, etc.).

Bonnes pratiques reconnues

- **Services singleton et DI** : Déclarer `AuthService` et l'intercepteur comme `@Injectable({providedIn: 'root'})` pour éviter les fuites mémoire. Regrouper la logique d'authentification dans ce service unique ¹.
- **Intercepteur global** : Enregistrez l'intercepteur dans `providers: [{ provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }]` (généralement dans le `CoreModule`). Cela permet d'ajouter automatiquement l'en-tête d'authentification et de gérer les 401 de façon centralisée ³ ¹³.
- **Expiration proactive** : Faire expirer les access tokens rapidement (p.ex. 15 min) et rafraîchir un peu avant l'expiration pour améliorer l'UX.
- **Route Guards** : Utiliser des `CanActivate` (via `AuthGuard`) pour protéger les routes sensibles, et vérifier côté serveur l'ID utilisateur (ne pas se fier qu'au front). Angular recommande d'avoir des guard type `canActivate` ou `canLoad` selon le besoin ¹⁴.
- **Sécurité des cookies** : Configurer toujours `Secure`, `HttpOnly` et un `SameSite` approprié sur les cookies d'authentification ⁵ ⁷. Éviter à tout prix de mettre le JWT en clair dans `localStorage`.
- **Logout propre** : Prévoir une méthode pour effacer le cookie de refresh (côté serveur) lors du logout. Par sécurité, actualiser/révoquer le refresh token en base de données côté backend à chaque renouvellement ⁸.
- **Bibliothèques matures** : On peut envisager des solutions éprouvées (OAuth2/OIDC via un BFF, des bibliothèques comme Auth0 ou SuperTokens) pour gagner en sécurité. La tendance (BFF pattern) est de ne **jamais exposer** les tokens au client, mais gérer les sessions côté serveur ¹⁵.

En résumé, la stratégie la plus sûre actuellement est de stocker le **refresh token** dans un cookie `HttpOnly` sécurisé (éventuellement `SameSite`) et de conserver l'**access token** en mémoire. L'intercepteur Angular ajoute alors l'`Authorization: Bearer ...` pour l'access token, et en cas de 401 il appelle `/api/auth/refresh` pour régénérer les tokens ⁸ ³. Ce schéma, couplé aux protections XSRF d'Angular et à des flags de cookie stricts, correspond aux bonnes pratiques recommandées par la communauté Angular et les experts en sécurité ¹⁶ ¹⁰.

Sources : Documentation Angular et guides de sécurité (ex. Angular University, StackOverflow, blogs techniques) ³ ⁸ ⁵ ⁶ ¹¹ ¹⁶.

1 2 6 7 **Angular Authentication With JWT: The Complete Guide**

<https://blog.angular-university.io/angular-jwt-authentication/>

3 **Angular - 使用 HTTP 与后端服务进行通信**

<https://v10.angular.cn/guide/http>

4 8 **angular - Storing JWT token into HttpOnly cookies - Stack Overflow**

<https://stackoverflow.com/questions/68777033/storing-jwt-token-into-http-only-cookies>

5 15 **angular - Best practises to store refresh token in cookie - Stack Overflow**

<https://stackoverflow.com/questions/73586668/best-practises-to-store-refresh-token-in-cookie>

9 11 12 13 14 16 **Angular Security Best Practices 2025 - Corgea - Home**

<https://corgea.com/Learn/angular-security-best-practices-2025>

10 **CSRF Protection in Angular: Secure Your Web Apps**

<https://www.stackhawk.com/blog/angular-csrf-protection-guide-examples-and-how-to-enable-it/>